UNIVERSITI TUNKU ABDUL RAHMAN
LEE KONG CHIAN FACULTY OF ENGINEERING AND SCIENCE
LABORATORY SAFETY RULES

The following rules must be obeyed by all students in the Computer laboratories of the faculty. Willful or repeated inadvertent noncompliance may result in dismissal or suspension from the laboratories.

I. No entry without permission:

i) Outsiders are not allowed to enter the laboratory without permission.

ii) No student is allowed to enter the laboratory unless permission has been given by a laboratory staff or a lecturer.

II. At work in the laboratory:

i) The computer facilities at the laboratory are to be used for learning purposes only.

ii) Wear your I.D. cards at all times in a proper visible position whenever you come into the computer laboratory.

iii) It is compulsory to follow dress code of UTAR
   a) Covered shoes (no slippers or flip flops)
   b) Long pants and tidy attire

iv) Do not eat or drink in the laboratory.

v) Do not tamper with any computers, printers, or any other associated equipment. If there are problems or faults, report immediately to the staff at the laboratory.

vi) Do not remove any equipment or document from the computer laboratories.

vii) Do not alter the default hardware and software setup of the computers and printers in the computer laboratories.

viii) Do not copy, rename, change, examine or delete files or information belonging to any other user of the University.

ix) Do not install any unauthorized or unlicensed software into the PC of computer laboratories.

x) Do not make any noise, chit-chat, hold group discussions or disturb other users at the laboratory in any way.

xi) Silent your hand phone when you are in the computer laboratories.

xii) Any equipment broken or damaged must be reported immediately to the laboratory staff.

III. Before leaving the laboratory:

i) No student is allowed to take away any equipment, tools or other property of the laboratory without permission.

ii) Please check and remember to take all your personal belongings such as handphone, wallet, pendrive, hard disk, book, bottle, sweeter, and etc.

iii) Lab staffs are not responsible for lost of personal belongings.

Universiti Tunku Abdul Rahman

Faculty	**Lee Kong Chian Faculty of Engineering and Science**
Department:	**Department of Electrical and Electronics Engineering**
Unit Code and Name	**UEEN 1043 Object-Oriented Concepts and Programming Techniques**
Experiment No.:	**7 / 7**
Title of Experiment:	**Overloading, Inheritance, and Polymorphism**
Laboratory Room No. and Name:	**Computer lab**
Experiment Duration (hour):	**3 hours**
Number of Student per Group	**1 student**

Objectives

- To practice the concept of overloading, inheritance, and polymorphism in OOP.

Equipment and Materials

Item Description	***Item category**	Quantity estimation (e.g. per set/group of student)
Personal computer	E	1 per student
Python interpreter (Anaconda Spyder)	S	1 per student

***Item category**	
SP	Sample or specimen
C	Consumable
CH	Chemical
W	Labware, glassware, tool, and components
E	Equipment
S	Software

Caution Statement (Optional)

FACULTY OF ENGINEERING AND SCIENCE
UNIVERSITI TUNKU ABDUL RAHMAN

UEEN 1043 Object-Oriented Concepts and Programming Techniques

Practical 7: Overloading, Inheritance and Polymorphism

Objectives

- To practice the concept of overloading, inheritance, and polymorphism in OOP.

Equipment

- Personal computer with a Python interpreter (Anaconda Spyder)

Introduction

Launch Anaconda Spyder IDE for Python and write codes for the following.

1 Classes

Python's convention to make an instance variable **protected** is to add a prefix '_' (single underscore) to it. **Protected** effectively prevents the instance variable to be accessed from outside of the class, like *private*, or unless the access is from within a sub-class. It is different from *private* which does not allow any accesses from both sub-classes and outside of the class. See the difference by comparing two snippets below:

snippet #1:

```
class employee:
    def __init__(self, name, sal):
        self.__name=name  # private attribute; use two underscores
        self.__salary=sal
class engineer(employee):
    def info(self):
        print('{}: salary is {}'.format(self.__name, self.__salary))
person = engineer('Bob', 4000)
person.info()
```

snippet #2:

```
class employee:
    def __init__(self, name, sal):
        self._name=name  # protected attribute; use one underscore
        self._salary=sal
class salesman(employee):
    def info(self):
        print('{}: salary is {}'.format(self._name, self._salary))
person = salesman('Steve', 8000)
person.info()
```

2 Operator overloading

Experiment the snippet below to understand the idea of **operator overloading** in Python.

```
import math
class Point:
    def __init__(self, xCoord, yCoord):
        self.__xCoord = xCoord
        self.__yCoord = yCoord
        self.__len = math.sqrt(xCoord ** 2 + yCoord ** 2)
        self.__angle = math.atan(yCoord / xCoord)

    def __str__(self): # overload the builtin function str()
        return ('point @ ({}, {}), |A| = {}, θ = {}'.format(
            self.__xCoord, self.__yCoord, self.__len, self.__angle))

    def __add__(self, y):
        x = self.__xCoord + y.__xCoord
        y = self.__yCoord + y.__yCoord
        return Point(x, y)

x = Point(math.sqrt(3), 3)
y = Point(1, 1)
print('x =', str(x))
print('y =', str(y))
z = x + y
print('z =', str(z))
```

3 Inheritance and Polymorphism

The following Python program processes `Student` and `Staff` records from *persons.csv* and organizes them using OOP principles.

(a) Inheritance

- **Base Class:** `Person` (General attributes like `ID`, `Name`, and `Age`).
- **Derived Classes:**
  i. `Student` (inherits Person, adds `Major` & `CGPA`).
  ii. `Staff` (inherits Person, adds `Major` & `YearsExperience`).

(b) Polymorphism: Each subclass **overrides** the `display_info()` method to provide specific details.

```python
# Base Class: Person
class Person:
    def __init__(self, person_id, name, age):
        self.person_id = person_id
        self.name = name
        self.age = int(age)

    def display_info(self):
        print(f"ID: {self.person_id}, Name: {self.name}, Age: {self.age}")

# Subclass: Student
class Student(Person):
    def __init__(self, student_id, name, age, major, cgpa):
        super().__init__(student_id, name, age)
        self.major = major
        self.cgpa = float(cgpa) if cgpa else 0.0  # Convert to float

    def display_info(self):
        print(f"ID: {self.person_id}, Name: {self.name}, Age: {self.age},
Role: Student, "
              f"Major: {self.major}, CGPA: {self.cgpa:.2f}")

# Subclass: Staff
class Staff(Person):
    def __init__(self, staff_id, name, age, major, years_experience):
        super().__init__(staff_id, name, age)
        self.major = major
        self.years_experience = int(years_experience) if years_experience
else 0  # Convert to int

    def display_info(self):
        print(f"ID: {self.person_id}, Name: {self.name}, Age: {self.age},
Role: Staff, "
              f"Major: {self.major}, Experience: {self.years_experience}
years")



# Function to load data from CSV file
def load_data(filename):
    people = []
    try:
        with open(filename, mode='r', encoding='utf-8') as file:
            lines = file.readlines()
            headers = lines[0].strip().split(",")  # Extract headers
            for line in lines[1:]:
                data = line.strip().split(",")  # Read each row
                person_data = dict(zip(headers, data))  # Map headers

                major = person_data["Major"]
                cgpa = person_data["CGPA"]
                years_experience = person_data["YearsExperience"]
```

```
                if cgpa:
                    person = Student(person_data["ID"],
person_data["Name"], person_data["Age"], major, cgpa)
                elif years_experience:
                    person = Staff(person_data["ID"], person_data["Name"],
person_data["Age"], major, years_experience)
                else:
                    person = Person(person_data["ID"],
person_data["Name"], person_data["Age"])

                people.append(person)
        print("Data successfully loaded from", filename)
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    return people

# Function to display all people
def display_people(people):
    print("\nPeople List:")
    for person in people:
        person.display_info()

# Function to get students with CGPA < 2.0
def get_students_below_2_cgpa(people):
    print("\nStudents with CGPA < 2.0:")
    for person in people:
        if isinstance(person, Student) and person.cgpa < 2.0:
            person.display_info()

# Function to get staff with ≥10 years experience
def get_staff_with_10_plus_years(people):
    print("\nStaff with >= 10 years of experience:")
    for person in people:
        if isinstance(person, Staff) and person.years_experience >= 10:
            person.display_info()

# Main Execution
people = load_data("persons.csv")
display_people(people)
get_students_below_2_cgpa(people)
get_staff_with_10_plus_years(people)
```